

```

        d[day+1][s2] = v;
        opt[day+1][s2] = o;
        prev[day+1][s2] = s;
    }
}

double dp() {
    for(int day = 0; day <= m; day++)
        for(int s = 0; s < states.size(); s++) d[day][s] = -INF;
    d[0][0] = c;
    for(int day = 0; day < m; day++)
        for(int s = 0; s < states.size(); s++) {
            double v = d[day][s];
            if(v < -1) continue;
            update(day, s, s, v, 0); //HOLD
            for(int i = 0; i < n; i++) {
                if(buy_next[s][i] >= 0 && v >= price[i][day] - 1e-3)
                    update(day, s, buy_next[s][i], v - price[i][day], i+1); //BUY
                if(sell_next[s][i] >= 0)
                    update(day, s, sell_next[s][i], v + price[i][day], -i-1); //SELL
            }
        }
    return d[m][0];
}

```

最后是打印解的部分。因为状态从前到后定义，因此打印解时需要从后到前打印，用递归比较方便。

```

void print_ans(int day, int s) {
    if(day == 0) return;
    print_ans(day-1, prev[day][s]);
    if(opt[day][s] == 0) printf("HOLD\n");
    else if(opt[day][s] > 0) printf("BUY %s\n", name[opt[day][s]-1]);
    else printf("SELL %s\n", name[-opt[day][s]-1]);
}

```

9.5 竞赛题目选讲

例题 9-18 跳舞机 (Tango Tango Insurrection, UVa 10618)

你想学着玩跳舞机。跳舞机的踏板上有 4 个箭头：上、下、下、右。当舞曲开始时，屏幕上会有一些箭头往上移动。当向上移动箭头与顶部的箭头模板重合时，你需要用脚踩

一下踏板上的相同箭头。不需要踩箭头时，踩箭头并不会受到惩罚，但当需要踩箭头时，必须踩一下，哪怕已经有一只脚放在了该箭头上。很多舞曲的速度快，需要来回倒腾步子，所以最好写一个程序来帮助你选择一个轻松的踩踏方式，使得能量消耗最少。

为了简单起见，将一个八分音符作为一个基本时间单位，每个时间单位要么需要踩一个箭头（不会同时需要踩两个箭头），要么什么都不需要踩。在任意时刻，你的左右脚应放在不同的两个箭头上，且每个时间单位内只有一只脚能动（移动和/或踩箭头），不能跳跃。另外，你必须面朝前方以看到屏幕（例如，你不能把左脚放到右箭头上，右脚放到左箭头上）。

当你执行一个动作（移动和/或踩）时，消耗的能量这样计算：

- ❑ 如果这只脚上个时间单位没有任何动作，消耗 1 单位能量。
- ❑ 如果这只脚上个时间单位没有移动，消耗 3 单位能量。
- ❑ 如果这只脚上个时间单位移动到相邻箭头，消耗 5 单位能量。
- ❑ 如果这只脚上个时间单位移动到相对箭头（上到下，或者左到右），消耗 7 单位能量。

正常情况下，你的左脚不能放到右箭头上（或者反之），但有一种情况例外：如果你的左脚在上箭头或者下箭头，你可以临时扭着身子用右脚踩左箭头，但是在你的右脚移出左箭头之前，你的左脚都不能移到另一个箭头上。类似地，右脚在上箭头或者下箭头时，你也可以临时用左脚踩右箭头。一开始，你的左脚在左箭头上，右脚在右箭头上。

输入包含最多 100 组数据，每组数据包含一个长度不超过 70 的字符串，即各个时间单位需要踩的箭头。L 和 R 分别表示左右箭头，“.”表示不需要踩箭头。输出应是一个长度和输入相同的字符串，表示每个时间单位执行动作的脚。L 和 R 分别是左右脚，“.”表示不踩。

【分析】

虽然本题的条件比较杂乱，但总的来说不难发现：可以按“箭头”划分阶段，再记录一下左右脚的位置以及上次左脚有没有踩，就可以顺利地动态规划了。

具体来说，用 $d(i, a, b, s)$ 表示已经踩了 i 个箭头 ($i \geq 0$)，左右脚分别在箭头 a 和 b 上，且上一个周期移动脚的集合为 s ($s=0$ 表示没有脚移动， $s=1$ 表示左脚移动， $s=2$ 表示右脚移动)，则最终答案为 $d(0, 1, 2, 0)$ 。4 个箭头的编号为 0-上，1-左，2-右，3-下。

如果下一步是“.”，有 3 种决策：左脚移动到另一个箭头；右脚移动到另一个箭头；不动。注意，虽然这次移动什么箭头都不会踩到，但还是要输出移动脚。

如果下一步是 4 个箭头之一，有两种决策：左脚移动到该箭头；右脚移动到该箭头。注意不要枚举不符合题目要求的移动方式。

例题 9-19 团队分组 (Team them up!, ACM/ICPC NEERC 2001, UVa1627)

有 n ($n \leq 100$) 个人，把他们分成非空的两组，使得每个人都分到一组，且同组中的人相互认识。要求两组的成员人数尽量接近。多解时输出任意方案，无解时输出 No Solution。

例如，1 认识 2, 3, 5；2 认识 1, 3, 4, 5；3 认识 1, 2, 5；4 认识 1, 2, 3；5 认识 1, 2, 3, 4（注意 4 认识 1 但 1 不认识 4），则可以分两组： $\{1, 3, 5\}$ 和 $\{2, 4\}$ 。

【分析】

设两个组的编号为 0 和 1。因为同组中的人相互认识，所以如果有两个人 a 和 b 不是相互认识，那么 a 和 b 只能分到两个不同的组。这样，如果已知某个人是第 0 组，那么不认

识它的所有人都应该是第1组。而不认识这些人的所有人都应该是0组，依此类推。这样，如果把“不相互认识”关系看成一个图，则每个连通分量都可以独立推导（推导过程中可能遇到矛盾，此时原问题无解）。例如，上面的样例对应图9-16（注意a认识b，但b不认识a，也应该连一条边）。

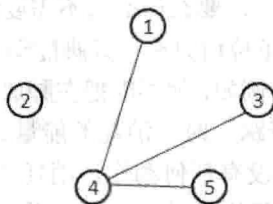


图 9-16 团队分组样例示意图

对于连通分量{1,3,4,5}，假设1在组0，可以推导出3,4,5都在组1；反过来，如果1在组1，可以推导出3,4,5都在组0。设组0比组1的人数多 d 个，可以总结出如表9-1所示。

表 9-1 组0和组1人数分布

	情况 1	情况 2
连通分量 1	组 0: {2}; 组 1: {} (d 加 1)	组 0: {}; 组 1: {2} (d 减 1)
连通分量 2	组 0: {4}; 组 1: {1,3,5} (d 减 2)	组 0: {1,3,5}; 组 1: {4} (d 加 2)

可以看到，每个连通分量的两种情况分别对应于 d 加一个值或者减一个值，最终目标是 d 的绝对值尽量少。想到了什么？没错！是0-1背包问题，只是没有“体积”，而“重量”有正有负，最后也不是要“重量”最大，而是最接近0。

例题 9-20 装满水的气球（Dropping water balloons, UVa 10934）

一年一度的新生周活动开始了，你们做好了大量的装满水的气球，准备拿来恶搞那些可怜的新生。活动开始之前，你们突然发现一个问题：这些气球实在是太硬了，很难把它们打破（如果打不破，它们就没有任何意义了）。甚至从好几层高的楼顶上把它们扔到地面，也打不破。你的任务是借助一个 n 层的高楼确定气球的硬度（所有气球硬度相同）。

实验过程是这样的：每次你拿着一个气球爬到第 f 层楼，将它摔到地面。如果气球破了，说明它的硬度不超过 f ；如果没破，说明硬度至少为 f 。注意，气球不会被实验所“磨损”。换句话说，如果在某层楼上往下摔，气球没破，那么在同一层楼不管再摔多少次它也不会破。

给你 k 个气球用来实验（可以打破它们）。你的任务是求出至少需要多少次实验，才能确定气球的硬度（或者得出结论：站在最高层也摔不破）。

输入每行包含两个整数 k, n （ $1 \leq k \leq 100, 1 \leq n < 2^{64}$ ），输出最少需要的实验次数。如果63次不够，输出“More than 63 trials needed”。

【分析】

用状态 $d(i, j)$ 表示用 i 个球实验 j 次所能测试的楼的最高层数。根据动态规划的常见思路，我们考虑第一次决策，设测试楼层为 k 。

如果气球破了，说明前 $k-1$ 层必须能用 $i-1$ 个球实验 $j-1$ 次测出来，也就是说，取 $k=d(i-1, j-1)+1$ 是最优的。

如果气球没有破,则相当于把第 $k+1$ 层楼看作 1 楼以后继续。因此在第 k 层楼之上还可以测 $d(i,j-1)$ 层楼,即 $d(i,j) = k + d(i,j-1) = d(i-1,j-1) + 1 + d(i,j-1)$ 。

例题 9-21 修缮长城 (Fixing the Great Wall, ACM/ICPC CERC 2004, UVa1336)

长城被看作一条直线段,有 n ($1 \leq n \leq 1000$) 个损坏点需要用机器人 GWARR 修缮。可以用三元组 (x_i, c_i, d_i) 描述第 i 个损坏点的参数,其中 x_i 是位置, c_i 是立刻修缮 (即时刻=0 时开始修缮) 的费用, d_i 是单位时间增加的修缮费用。换句话说,如果在时刻 t_i 开始修缮第 i 个损坏点,费用为 $c_i + t_i d_i$ 。上述参数满足 $1 \leq x_i \leq 500000$, $0 \leq c_i \leq 50000$, $1 \leq d_i \leq 50000$ 。

修缮的时间忽略不计, GWARR 的速度恒定为 v ($1 \leq v \leq 100$), 因此从修缮点 i 走到修缮点 j 需要 $|x_i - x_j|/v$ 单位的时间。初始坐标为 x ($1 \leq x \leq 500000$)。输入保证损坏点的位置各不相同,且 GWARR 的初始位置不与任何一个损坏点重合。

你的任务是找到修缮所有点的最小费用 (用截尾法保留整数部分)。输入保证最小费用不超过 10^9 。

【分析】

首先将所有修缮点按照坐标从小到大排序,不难发现在任意时候,已修复的点一定是一个连续的区间,因此可以考虑用 $d(i,j,k)$ 表示修复完 (i,j) ,且当前位置为 k ($k=0$ 表示在左端点 i , $k=1$ 表示在右端点 j) 时已经发生的总费用。

但是这样会带来一个问题:今后的费用无法计算,因为不知道当前时间。不过没关系,谁说必须当费用发生以后才能计算?可以事先把还没有发生但是肯定会发生的费用累加到答案中,然后“时钟归零”。事实上,在前面已经用过一次这种技巧了,那就是例题“颜色的长度”。

设 $d(i,j,k)$ 表示修复完 (i,j) ,且当前位置为 k (含义同上) 时,已经发生的总费用与所有“肯定会发生的未来费用”之和,使用刷表法,则一共只有两个决策。

决策 1: 往左走,修理点 $i-1$, 转移到 $d(i-1,j,0)$ 。假设当前点为 p ($k=0$ 时 $p=i$, 否则 $p=j$), 则到达点 $i-1$ 的时间为 $t = |X_{i-1} - X_p|/v$ 。在这段时间里,所有未修理点 (即点 $1 \sim i-1$ 和 $j+1 \sim n$) 的费用都增加了 t ,需要把这些点的总费用 $(\text{sum_d}(1,i-1) + \text{sum_d}(j+1,n)) * t$ 累加到状态值中,然后点 $i-1$ 的修理费用就只有 c_{i-1} 了。即用 $d(i,j,k) + (\text{sum_d}(1,i-1) + \text{sum_d}(j+1,n)) * t + c_{i-1}$ 来更新 $d(i-1,j,0)$ 。其中 $\text{sum_d}(i,j)$ 表示点 $i \sim j$ 的所有 d 值之和。

决策 2: 往右走,修理点 $j+1$, 转移到 $d(i,j+1,1)$ 。和决策 1 很类似,方程略。

状态有 $O(n^2)$ 个,每个状态只有两个决策,因此时间复杂度为 $O(n^2)$ 。

例题 9-22 越大越好 (Bigger is Better, ACM/ICPC Xi'an 2006, UVa12105)

你的任务是用不超过 n ($n \leq 100$) 根火柴摆一个尽量大的,能被 m ($m \leq 3000$) 整除的正整数。例如, $n=6$ 和 $m=3$, 解为 666。无解输出 -1, 如图 9-17 所示。

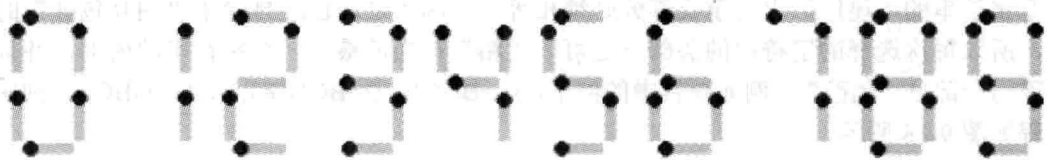


图 9-17 火柴数字

【分析】

一般来说,整数是从左往右一位一位写的,因此不难想到这样的动态规划算法:用 $d(i,j)$ 表示用 i 根火柴能拼出的“除以 m 余数为 j ”的最大数,然后用刷表法,枚举在最右边添加的数字 k ,用 $d(i,j)*10+k$ 更新 $d(i+c(k), (j*10+k)\%m)$,其中 $c(k)$ 表示数字 k 需要的火柴数。状态有 $O(nm)$ 个,每个状态只有“在右边添加数字 0~9”这 10 个决策,看上去不错。可惜这个算法有个缺点:状态值是高精度整数,因此实际计算量比较大。

还有一个算法,虽然有些难想,但是效率很高:用 $d(i,j)$ 表示拼出一个“除以 m 余数为 j 的 i 位数”至少需要多少火柴(若无解, $d(i,j)$ 为正无穷)。状态转移方程和上面类似,留给读者思考。因为此处只关心位数,这个算法并不涉及高精度整数。

如何根据 $d(i,j)$ 计算出题目要求的答案呢?首先确定最大的位数 w (即让 $d(i,0)$ 不是正无穷的最大 i),因为位数越大,整数就越大(不允许有前导 0,因为不划算)。接下来从左到右依次确定各个数字。

例如,假定 $m=7$,并且已经确定最大的整数是 3 位数。首先试着让最高位为 9。如果可以摆出形如 9ab 的整数,它一定是最大的。是否可以摆出 9ab 呢?因为 900 除以 7 的余数为 4,后两位“ab”除以 7 的余数应为 3。如果 $d(2,3)+c(9)\leq n$,说明火柴足够摆出 9ab,否则说明最高位不能是 9。重复这个过程,直到所有数字都被确定为止。这个过程需要快速算出形如 $x000\cdots$ 的整数除以 m 的余数,可以通过一个预处理完成,留给读者思考。

例题 9-23 有趣的游戏 (Fun Game, ACM/ICPC Beijing 2004, UVa1204)

一些小孩围成一圈做游戏。每一轮从某个小孩开始往他左边或右边传手帕。一个小孩拿到手帕后(包括第一个小孩)在手帕上写下自己的性别,男孩写 B,女孩写 G,然后按相同方向传给下一个小孩,每一轮可能在任何一个小孩写完后停止。现在游戏已经进行了 n 轮,已知 n 轮中每轮手帕上留下的字,求最少可能有几个小孩。 $2\leq n\leq 16$ 。每轮手帕上的字数不超过 100。

例如,若 3 轮的手帕上分别留下 BGGB, BGBGG, GGGBGB,则至少有 9 个小孩。一种可能性是 GGGBGBGB。

【分析】

首先可以看出,如果有一个字符串完全包含于其他某个字符串,那么这个字符串将对结果没有影响,所以先预处理去掉这些字符串。后面将看到这会给动态规划带来方便。

在解决原题之前,先看一个简化版:小孩排成一行(而不是一圈),且传递手帕总是从左到右的。那么问题就等价于:找一个最短的字符串,使得输入的 n 个字符串都是它的连续子串。

可以把这个问题转化为一个多阶段决策过程:每次选择一个字符串“粘”在当前最后一个字符串的“尾巴”上(重叠部分必须相等)。因为之前已经排除了“相互包含”的情况,所以每次选择的字符串的头部一定可以“粘”在当前最后一个字符串的内部,并且可以露出一部分“尾巴”。例如题目中的例子, $s_1=BGGB$, $s_2=BGBGG$, $s_3=GGGBGB$,则决策过程如图 9-18 所示。

最终得到的字符串长度等于所有 n 个字符串的长度之和,减去每个串(除了第一个串)与前一个串的最大重叠长度。对于上面的例子, s_1, s_2, s_3 的长度之和为 15, s_2 和 s_3 的最大

重叠长度为 3, s_1 和 s_2 的最大重叠长度为 3, 因此最终得到的字符串长度为 $15-3-3=9$ 。注意上述“最大重叠长度”不是对称的, 例如, 若 s_2 在右边, s_2 和 s_3 可以重叠 3 个字符, 但如果 s_2 在左边, 则只能重叠 2 个字符。

这个过程启发我们使用动态规划。用 $d(i, j)$ 来表示已经选过的字符串集合为 i , 最后一个串为 j 时, 可以减去的重叠部分总长。如图 9-19 所示, 假设已经选择了字符串 1, 6, 4, 其中最后一个字符串为 4, 即状态 $d(\{1, 4, 6\}, 4)$ 。假设接下来选择字符串 3, 并且已经得到了 3 粘在 4 尾巴上时的最大重叠长度为 5, 则可以用 $d(\{1, 4, 6\}, 4)+5$ 来更新 $d(\{1, 3, 4, 6\}, 3)$ 。

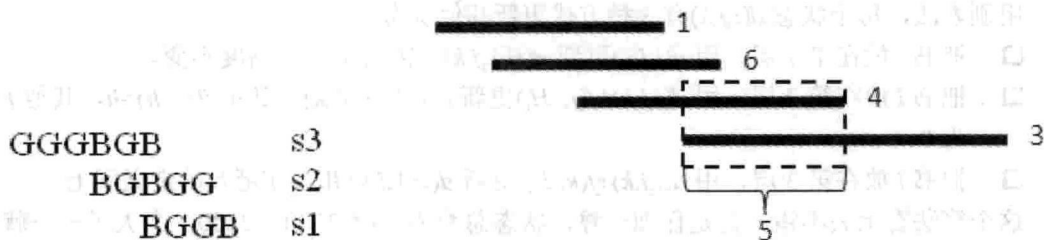


图 9-18 决策过程

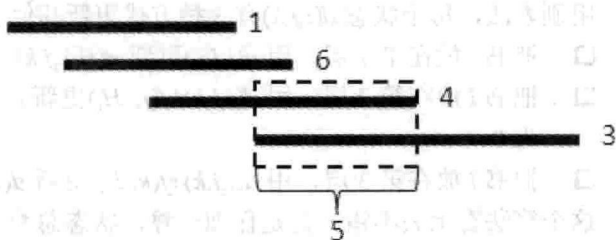


图 9-19 已选字符串 1, 6, 4 的情况

现在已经解决了简化版问题, 原题只有两点不同:

(1) 原题中, 手帕有两种不同的方向, 因此选择每个串之后, 还要确定是把它直接粘上呢, 还是反过来粘, 因此状态 $d(i, j)$ 中的 j 有 $2n$ 种可能, 每次的决策也变成 $2n$ 个, 时间复杂度不变, 只是常数略有增加。

(2) 原题中, 所有小孩组成一个圈, 因此需要考虑如何把链变成圈。一种方法是在状态中增加一维, 用来记录第一个串是哪个, 这样就可以在最后一次决策时计算最后一个串和第一个串的公共部分。这样做并没有错, 但是因为状态多了一维, 时间复杂度也将变大。其实, 第一个串和最后一个串并不一定会重叠。如果是那样, 按链的方式计算出的就是正确答案; 如果第一个串和最后一个串重叠, 但是中间某个串 i 和下一个串不重叠, 链的答案仍然是正确答案, 因为可以在串 i 后面把圈“打断”成链, 让 i 成为“最后一个串”。如果每个串和后一个串都有重叠怎么办? 那样只能按照圈的做法了, 不过任选一个串作为第一个串都行, 不用给状态增加一维。另外还有一个地方要注意: 输入字符串不一定是圈的一部分, 它可能绕了好几圈(想一想, 应如何处理)。

这样, 即把简化版问题的解扩展成了原题的解法, 时间复杂度仍是 $O(n^2 \cdot 2^n)$ 。

例题 9-24 书架 (Bookcase, ACM/ICPC NWERC 2006, UVa12099)

有 n ($3 \leq n \leq 70$) 本书, 每本书有一个高度 H_i 和宽度 W_i ($150 \leq H_i \leq 300$, $5 \leq W_i \leq 30$)。现在要构建一个三层的书架, 你可以选择将 n 本书放在书架的哪一层。设三层高度 (该层书的最高高度) 之和为 h , 书架总宽度 (即每层总宽度的最大值) 为 w , 则要求 $h \cdot w$ 尽量小。

【分析】

如果所有书的高度都相等, 本题就是“分成 3 个子集, 使得元素和的最大值尽量小”, 而这是 0-1 背包类型的问题。这提示我们需要把宽度写到状态里。

首先将所有的书按照高度从大到小排序。不妨设高度最大的书安排在第 1 层, 且第 2 层的高度大于等于第 3 层的高度, 然后设状态 $d(i, j, k)$ 表示安排完前 i 本书, 第 2 层书的宽度

之和为 j ，第 3 层书的宽度之和为 k 时，第 2 层高度和第 3 层高度和的最小值。

为什么不记录第 1 层的高度？因为最高的书在第 1 层，意味着这一层永远都不会比它更高了；为什么不记录第 1 层的宽度？因为目前 3 层的总宽度等于前 i 本书的总宽度，只要知道了第 2、3 层的宽度，就能算出第 1 层的宽度。另外，因为这些书已经按照高度从大到小排序了，一旦 3 层都放了书，3 层的高度都不会变了，因此：

- 如果只有前两层放了书，当且仅当往第 3 层放书 i 时，第 3 层高度会从 0 变到 H_i 。
 - 如果只有第 1 层放了书，当且仅当往第 2 层放书 i 时，第 2 层高度会从 0 变到 H_i 。
- 用刷表法，每个状态 $d(i, j, k)$ 有 3 种方式更新其他状态：
- 把书 i 放在第 1 层，用 $d(i, j, k)$ 更新 $d(i+1, j, k)$ ，因为第 1 层高度不变。
 - 把书 i 放在第 2 层，用 $d(i, j, k) + f(j, H_i)$ 更新 $d(i+1, j+W_i, k)$ ，其中 $f(0, h) = h$ ，其他 f 值为 0。
 - 把书 i 放在第 3 层，用 $d(i, j, k) + f(k, H_i)$ 更新 $d(i+1, j, k+W_i)$ ， f 函数的定义同上。

这个算法看上去不错，但是仔细一算，状态总数为 $70 * 2100 * 2100$ ，太大了——就算作用时间能接受，所占用的空间也无法接受，因此无法使用记忆化搜索，而只能用递推，配合滚动数组（由于是 0-1 背包式的递推， i 那一维可以完全省略）。

如何优化呢？出乎大多数选手的意料^①，本题的“标准优化”并没有降低理论时间复杂度，只是让程序的实际运行效率高了很多。优化有两种：

- $j+k$ 不应该超过前 i 本书的宽度之和，因此有用的状态比 $70 * 2100 * 2100$ 少得多。
- 假设第 i 层书的总宽度为 ww_i ，如果 $ww_2 > ww_1 + 30$ （30 是一本书的宽度上限），那么可以把第 2 层的一本书放到第 1 层来，则前两层高度之和不会变大，书架宽度（即两层总宽度的最大值）也不会变大。因此，只需要计算满足 $ww_2 \leq ww_1 + 30$ 且 $ww_3 \leq ww_2 + 30$ 的状态，因此 $j \leq (2100 + 30) / 2 = 1065$ ， $k \leq (2100 + 60) / 3 = 720$ 。

强烈建议读者实现优化前后的两个版本，比较二者的效果。

例题 9-25 轻松爬山 (Easy Climb, NWERC 2008, UVa12170)

输入正整数 d 和 n 个正整数 $h_1, h_2, \dots, h_n$ ，可以修改除了 h_1 和 h_n 的其他数，要求修改后相邻两个数之差的绝对值不超过 d ，且修改费用最小。设 h_i 修改之后的值为 h'_i ，则修改费用为 $|h_1 - h'_1| + |h_2 - h'_2| + \dots + |h_n - h'_n|$ 。无解输出 -1。 $N \leq 100$ ， $d \leq 10^9$ 。

【分析】

本题是一个多阶段决策过程：依次确定每个 h_i 修改成什么数。可惜 d 的范围太大，如果用 $f(i, x)$ 表示已经修改 i 个数，其中第 i 个数改成 x 时还需要的最小费用，则状态总数高达 $O(nd)$ 。

为了更好地分析问题，先来看看简化版： $n=3$ 时，只有 h_2 是可以修改的，而且修改之后必须同时在 $[h_1-d, h_1+d]$ 和 $[h_3-d, h_3+d]$ 内，即 $[\max(h_1, h_3)-d, \min(h_1, h_3)+d]$ 。如果这个区间是空的，说明无解；否则 h_2 要么不变，要么改成 $\max(h_1, h_3)-d$ 或者 $\min(h_1, h_3)+d$ 。

这个例子至少说明了：修改后的值并不是随便选的，至少在 $n=3$ 时，修改后的值只有 3 种选择： h_2 ， $\max(h_1, h_3)-d$ 和 $\min(h_1, h_3)+d$ 。

^① 本题的解法看上去比较常规，但是在 NWERC 这样较高水平的比赛中，却没有队伍做出来。

用类似的推理,可以得到这样的结论:每个数在修改之后一定可以写成 $h_p + kd$, 其中 $1 \leq p \leq n$, $-n < k < n$, 这样, 上述状态 $f(i, x)$ 中的 “ x ” 就只有 $O(n^2)$ 种可能了, 状态总数为 $O(n^3)$ 。

不难写出状态转移方程: $f(i, x) = |h_i - x| + \min \{f(i-1, y) \mid x-d \leq y \leq x+d\}$ 。如果按照 x 从小到大的顺序计算, 满足 $x-d \leq y \leq x+d$ 的 $f(i-1, y)$ 就是 $i-1$ 阶段状态值序列的一个滑动窗口。使用前面介绍过的单调队列, 可以在平摊 $O(1)$ 的时间复杂度内计算出 $f(i, x)$, 因此本题的总时间复杂度为 $O(n^3)$ 。

例题 9-26 一个调度问题 (A Scheduling Problem, ACM/ICPC Kaoshiung 2006, UVa1380)

有 n ($n \leq 200$) 个恰好需要一天完成的任务, 要求用最少的时间完成所有任务。任务可以并行完成, 但必须满足一些约束。约束分有向和无向两种, 其中 $A \rightarrow B$ 表示 A 必须在 B 之前完成, $A-B$ 表示 A 和 B 不能在同一天完成。输入保证约束图是将一棵 n ($n \leq 200$) 个结点的树的某些边定向后得到的。例如, 图 9-20 表示 1 和 2 不能在同一天完成, 1 必须在 3 之前, 3 必须在 5 之前, 2 必须在 4 之前, 4 必须在 6 之前。

可以使用如下定理: 忽略无向边之后, 设图上的最长链 (即包含点数最多的路径) 包含 k 个点, 则答案为 k 或者 $k+1$ 。对于上面的例子, 忽略无向边后的最长链是 $2 \rightarrow 4 \rightarrow 6$, 包含 3 个结点。

【分析】

如果树中所有边都为有向边, 那么答案就是最长链上的点数: 先将度为 0 的点全部安排在第一, 将这些点删去, 然后将新的度为 0 的点安排在第二天, 这样就可以在 k 天内安排完。这样, 原问题转化为: 将树中的所有无向边定向, 使得树中的最长链最短。

根据题目中的定理, 设原图中有向边组成的最长链上有 k 个点, 那么最终的答案不是 k 就是 $k+1$, 接下来只需要判断是否可以通过无向边定向使得最长链的点数为 k 。即使没有题目中的那个定理, 也可以二分答案 x , 然后判断是否能让最长链的点不超过 x 。不管是哪种情况, 问题的关键就是: 给定一个 x , 判断是否可以给无向边定向, 使得最长链点数不超过 x 。

设 $f(i)$ 表示以 i 为根的子树内的边全部定向后, 最长链点数不超过 x 的前提下, 形如 “后代到 i ” (如图 9-21 中的 $u' \rightarrow u \rightarrow i$) 的最长链的最小值, 同理可以定义 $g(i)$ 表示形如 “ i 到后代” (如图 9-21 中的 $i \rightarrow v \rightarrow v'$) 的最长链的最小值。达不到的状态 (即 “最长链点数不超过 x ” 这个前提无法满足) 定义为正无穷。

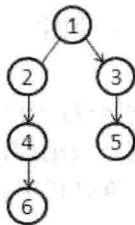


图 9-20 调度问题示意图

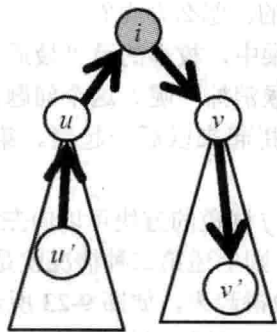


图 9-21 最长链

如何计算 $f(i)$ 和 $g(i)$ 呢? 为了叙述方便, 用 w 表示 i 的某个子结点, 则 w 和 i 之间的边有 3 种情况: $w \rightarrow i$, $i \rightarrow w$ 和 $i-w$, 其中前两种是有向边, 最后一种是无向边。按照从易到难的顺序, 分两种情况讨论。

情况 1: 如果 i 与 w 的所有边都是有向边, 直接计算即可。令 $f(i)$ 等于形如 $w \rightarrow i$ 的 w 的 $f(w)$ 的最大值加 1, $g(i)$ 等于形如 $i \rightarrow w$ 的 w 的 $g(w)$ 的最大值加 1, 则以 i 为根的子树内, 经过 i 的最长链点数等于 $f(i)+g(i)$ 。如果这个值大于 x , 则 $f(i)$ 和 $g(i)$ 为正无穷, 否则 $f(i)=f(i)$, $g(i)=g(i)$ 。

情况 2: 如果 i 与某些 w 之间存在无向边, 则需要确定每条 $i-w$ 定向成 $w \rightarrow i$ 还是 $i \rightarrow w$ 。由于定向完成之后, 仍需要按照情况 1 的方法计算, 所以问题的关键是分析“定向”操作会如何影响 $f(i)$ 和 $g(i)$ 。

求 $f(i)$ 时, 目标是 $f(i)+g(i) \leq x$ 的前提下 $f(i)$ 最小。首先把所有没定向的 $f(w)$ 从小到大排序。假定把 f 值第 p 小的 w 定向为 $w \rightarrow i$, 那么最好“顺便”把前 p 小的全部变成 $w \rightarrow i$ 的, 因为这样做不会让 $f(i)$ 变大, 但有可能让 $g(i)$ 变小, 百利而无一害。所以只需要枚举 p , 把 f 值前 p 小的 w 都定向为 $w \rightarrow i$, 其他定向为 $i \rightarrow w$, 然后计算 $f(i)$ 。用相同的方法可以计算 $g(i)$ 。最后判断根结点的 f 值是否无穷大即可。

例题 9-27 方块消除 (Blocks, UVa10559)

有 n ($n \leq 200$) 个带颜色方格排成一列, 相同颜色的方块连成一个区域。游戏时, 可以任选一个区域消去。设这个区域包含的方块数为 x , 则将得到 x^2 个分值, 然后右边所有方块就会向左移一格。如图 9-22 所示是一个游戏局面和最优消除方式。

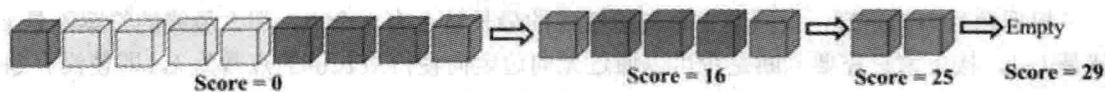


图 9-22 游戏局面和最优消除方式

你的任务是求出最高可能的得分。

【分析】

为了叙述方便, 设左数第 i 个方块的颜色为 $A[i]$ 。按照线性结构动态规划的常见思路, 设 $d(i,j)$ 表示子序列 $i \sim j$ 的最大得分, 但是似乎无法用 $d(i,k)$ 和 $d(k,j)$ 来计算 $d(i,j)$, 因为可能 $i \sim k$ 和 $k \sim j$ 各剩下一些, 拼起来以后消除。如 XAXBXCXDXEX, 实际上是把 A 和 E 全部单个消除以后再消除 X 的。怎么办呢?

在最优矩阵链乘中, 枚举的是“最后一次乘法”的位置。本题是不是也可以枚举“最后一个方块什么时候消掉”呢? 这个问题的答案有两种可能: 直接把它所在的一段消掉; 把它和左边的某段拼起来以后一起消。第一种情况容易处理, 但第二种情况就没那么简单了。

具体来说, 设与 j 同色的方块可以向左延伸到 p (即 $A[p]=A[p+1]=\dots=A[j]$), 且 $A[q]=A[j]$, $A[q]$ 不等于 $A[q+1]$, 则上述第二种情况就是指先把 $q+1 \sim p-1$ 这一段消掉, 把 $p \sim j$ 这一段和以 q 为右端点的那一段拼起来, 如图 9-23 所示。注意 $i \sim j$ 全部同色时找不到这样的 q , 但此时可以直接计算出结果。下面忽略这种情况。

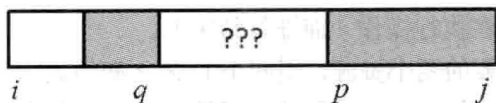


图 9-23 消掉与拼接方块

不过,把这两段拼起来以后仍然不一定立刻消除,还可能要和更左边的另一段拼起来……是不是很复杂?但有一点是可以肯定的,那就是 $q+1 \sim p-1$ 这一段肯定可以先消掉(拖到后面再消也得不到什么好处)。那么现在就把它消掉(得分是 $d(q+1, p-1)$),得到一个“子序列 $i \sim q$ 的右边再拼上 $j-p+1$ 个与 $A[q]$ 同色的方块”的奇怪状态,如图 9-24 所示。

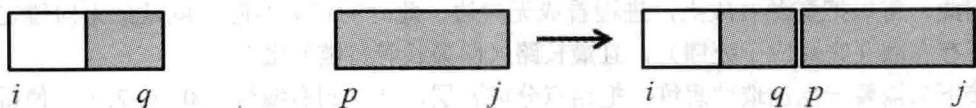


图 9-24 消掉后的奇怪状态

由此可知,在状态中增加一维,来表达“右边拼上一些方块”,即用 $d(i, j, k)$ 表示“原序列中的方块 $i \sim j$ 右边再拼上 k 个颜色等于 $A[j]$ 的方块所得到的新序列”的最大得分,则决策有两种。

决策 1: 直接消去方块 j , 转移到 $d(i, p-1, 0) + (j-p+1)^2$ 。

决策 2: 枚举 $q < p$ 使得 $A[q] = A[j]$ 且 $A[q] \neq A[q+1]$, 转移到 $d(q+1, p-1, 0) + d(i, q, j-p+1)$ 。

状态有 $O(n^3)$ 个, 决策有 $O(n)$ 个, 时间复杂度为 $O(n^4)$ 。如果采用记忆化搜索, 很多状态都达不到, 而且 q 的取值范围往往很小, 所以对于大部分数据, 这个算法的运行效率都很高。

例题 9-28 独占访问 2 (Exclusive Access 2, ACM/ICPC NEERC 2009, UVa1439)

在一个庞大的系统里运行着 n ($1 \leq n \leq 100$) 个守护进程。每个进程恰好用两个资源。这些资源不支持并发访问, 所以这些进程通过锁来保证互斥访问。每个进程的主循环如下:

```
loop forever
  DoSomeNonCriticalWork()
  P.lock()
  Q.lock()
  WorkWithResourcesPandQ()
  Q.unlock()
  P.unlock()
end loop
```

注意, P 和 Q 的顺序是至关重要的。如果某进程用到了消息队列和数据库, “先获取数据库的锁”与“先获取消息队列的锁”可能会产生截然不同的效果。给定每个进程所需要的两种资源, 你的任务是确定每个进程获取锁的顺序, 使得进程永远不会死锁, 且最坏情况下, 等待链的最大长度最短。

在本题中, 一个长度为 n 的等待链是一个不同资源和不同进程的交替序列: $R_0 c_0 R_1 c_1 \dots R_n c_n R_{n+1}$, 其中进程 c_i 已经获取 R_i 的锁, 正在等待 R_{i+1} 的锁。当 $R_0 = R_{n+1}$ 时死锁, 否则说明

已获取 R_{n+1} 的锁的进程正在执行操作（而非等待中）。

输入 n 和每个进程需要的两个资源，用两个 $L \sim Z$ 之间的大写字母表示（因此一共有 15 种资源）。输出包含两行，第一行为最坏情况下等待链的最大长度 m ，以下 n 行每行输出两个字符，表示该进程获取锁的顺序（先获取第一个字符对应资源的锁）。

【分析】

本题初看起来毫无头绪，甚至连数学模型都难以建立。注意，每个进程恰好需要两个资源，而等待链的定义是资源和进程的交替序列，可以联想到图论中的概念：每条边恰好连接两个点，路径的定义是点和边的交替序列。

因此，可以把资源看成点，进程看成无向边，此时的任务实际上就是把无向边定向，使得不存在圈（它对应于死锁），且最长路（即最长等待链）最短。

接下来需要一点创造性思维：把结点分成 p 层，从左到右编号为 $0, 1, 2, \dots$ ，使得同层结点之间没有边。对于任意一条边 $u-v$ ，把它定向成“从层编号小的点指向层编号大的点”。例如，若 u 在第 5 层， v 在第 2 层，则定向为 $v \rightarrow u$ 。定向之后的有向图肯定没有圈，且最长路包含的点数不超过 p （想一想，为什么），所以直观上， p 应该是越小越好。

事实上，可以证明^①当 p 取最小值时，最长路恰好包含 p 个结点，而且这个结果是所有定向方案中最优的。这样，就成功地把问题转化为了“结点分层”问题，而这个“结点分层”问题实际上就是之前学过的色数问题：把图中的结点染成尽量少的颜色，使得相邻结点颜色不同。套用前面学过的动态规划算法，在 $O(3^k)$ 时间内即解决了问题，其中 $k \leq 15$ ，为资源的最大数目。

本题是关于“建模与问题转换”的一道经典问题，请读者仔细体会。

例题 9-29 整数传输 (Integer Transmission, ACM/ICPC Beijing 2007, UVa1228)

你要在一个仿真网络中传输一个 n 比特的非负整数 k 。各比特从左到右传输，第 i 个比特的发送时刻为 i 。每个比特的网络延迟总是为 $0 \sim d$ 之间的实数（因此从左到右第 i 个比特的到达时刻为 $i \sim i+d$ 之间）。若同时有多个比特到达，实际收到的顺序任意。求实际收到的整数有多少种，以及它们的最小值和最大值。例如， $n=3$ ， $d=1$ ， $k=2$ （二进制为 010）时实际收到的整数的二进制可能是 001(1)、010(2)和 100(4)。 $1 \leq n \leq 64$ ， $0 \leq d \leq n$ ， $0 \leq k < 2^n$ 。

【分析】

为了简化问题，首先可以规定：所有 0 按照原来的顺序依次收到，所有的 1 也按照原来的顺序依次收到，只是 0 和 1 可能交错。这个规定非常重要，请读者仔细体会。

最小值和最大值可以用贪心法得到（留给读者思考），关键在于统计可能收到的整数数目。给定一个整数 P ，如何判断它是否可能被收到呢？来看一个例子。

例如， $k=11001010$ ， $d=3$ ，需要判断 $P=00111001$ 是否可以得到。一共有 8 个比特，则发送时刻为 $1 \sim 8$ ，接收时刻是 $1 \sim 12$ 。不难发现，接收时刻可以限制为 $1 \sim 8$ ，因为同一时刻接收的比特可以任意排列，所以把一个比特延迟到时刻 $9 \sim 12$ 不会有任何好处。可以手算出一种方案，如图 9-25 所示。

^① 证明思路是从定向方案构造分层图。先把所有路径的起点作为第 0 层。

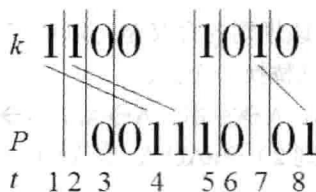


图 9-25 手算方案

上图的意思是： k 的比特 1 和比特 2 均延迟到时刻 4，比特 7 延迟到时刻 8。不难发现，对于任意给定的 P ，都可以用贪心法求解：从左到右依次考虑 P 的每一个比特。如果是 0，则接收 k 中没有收到的最左边的 0；如果是 1，则接收 k 中没有收到的最左边的 1。

仔细推敲这个过程，可以得到一个结论：在任意时刻， k 中已收到的比特中最右边的那个比特一定没有延迟（理论上可以延迟，但不会得到更优的解）。如图 9-26 所示， $k=111011001110$ ，框中的比特是已收到的比特，则最右边那个已收到比特（即左数第 3 个 0）无延迟，即接收时刻和发送时刻均为 8。

这样就可以动态规划了^①：用 $d(i, j)$ 表示 k 的前 i 个 0 和前 j 个 1 收到以后可能形成的整数个数，则只有两种转移方式：

如果下一个收到的比特可以是 0，则 $d(i+1, j)$ 需要加上 $d(i, j)$ 。

如果下一个收到的比特可以是 1，则 $d(i, j+1)$ 需要加上 $d(i, j)$ 。

所以问题的关键就是：如何判断下一个收到的比特是否可以为 0 或者 1？还是刚才那个例子，因为已经收到了 3 个 0 和 4 个 1，所以状态是 $d(3, 4)$ 。假设下一个收到的比特是 0，则左数第 5 个 1（发送时刻为 6）至少得延迟到第 4 个 0 的发送时刻（即时刻 12），如图 9-27 所示。如果 $d < 12 - 6 = 6$ ，说明假设不成立。

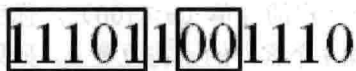


图 9-26 已收到的比特



图 9-27 判断收到的比特

一般地，设第 i 个 0 的发送时刻为 Z_i ，第 i 个 1 的发送时刻为 O_i ，则当且仅当 $O_j + d \leq Z_{i+1}$ 时 $d(i, j)$ 可以转移到 $d(i+1, j)$ ，即下一个收到的比特为 0。同理，当且仅当 $Z_i + d \geq O_{j+1}$ 时， $d(i, j)$ 可以转移到 $d(i, j+1)$ 。

状态有 $O(n^2)$ 个，每个状态只有两个决策，因此总时间复杂度为 $O(n^2)$ 。

例题 9-30 给孩子起名 (The Best Name for Your Baby, ACM/ICPC Yokohama 2006, UVa1375)

给一个包含 n 条规则的上下文无关文法和长度 l ($1 \leq n \leq 50$, $0 \leq l \leq 20$)，求出满足该文法的串中，长度恰好为 l 的字典序最小串。如果不存在，输出单个字符 “-”。

“满足文法”是指可以不断使用规则，把单个大写字母 S 变成这个串。每条规则形如 $A \rightarrow a$ ，其中 A 是一个大写字母（表示非终结符）， a 是一个由大小写字母组成的字符串（长

^① 准确地说这不是动态规划，而是组合数学中的递推，因为本题不是最优化问题，而是计数问题。不过解决两个问题的思路是相同的，所以很多人把组合数学中的递推也算作动态规划。

度不超过 10, 且可以为空串)。该规则的含义是可以字符串 a 来替换当前字符串中的大写字母 A (如果有多个 A , 每次只替换一个)。

例如, 有 4 条规则: $S \rightarrow aAB$, $A \rightarrow \text{空串}$, $A \rightarrow Aa$, $B \rightarrow AbbA$, 那么 $aabb$ 满足该文法, 因为 $S \rightarrow aAB$ (规则 1) $\rightarrow aB$ (规则 2) $\rightarrow aAbbA$ (规则 4) $\rightarrow aAabbA$ (规则 3) $\rightarrow aAabb$ (规则 2) $\rightarrow aabb$ (规则 2)。

【分析】

题目中的文法比较复杂, 首先把它们简化一下。例如 $S \rightarrow ABaA$ 拆成 3 个: $S \rightarrow AP_1$, $P_1 \rightarrow BP_2$, $P_2 \rightarrow aA$ 。这样, 所有规则都变成了 $A \rightarrow BC$ 的形式, 规则总数不超过 $50 \times 10 = 500$ 。为了叙述方便, 文法拆分后的所有大小字母和小写字母统称为符号。

接下来试着动态规划: $d(i, L)$ 表示符号 i 能变成的、长度为 L 且字典序最小的串。如果符号 i 不能变成长度为 L 的串, 则 $d(i, L)$ 无定义。例如, 符号 i 是小写字母且 L 不等于 1 时, $d(i, L)$ 无定义。是不是可以这样转移呢:

$$d(i, L) = \min\{d(j, p) + d(k, L-p) \mid \text{存在规则 } i \rightarrow jk, 0 \leq p \leq L\}$$

逻辑上没问题, 但是如果直接写一个记忆化搜索, 程序可能会无限递归: 如果有两个规则 $A \rightarrow BC$, $B \rightarrow AC$, 则 $d(A, L)$ 的计算需要调用 $d(B, L)$, 而计算 $d(B, L)$ 时又会调用 $d(A, L)$ ……

怎么办呢? 注意到上述情况只有 $p=0$ 或者 $p=L$ 时才会出现, 大多数情况下还是可以按照 L 从小到大的顺序计算的。所以可以特殊处理 L 相同时的所谓“同层状态转移”。

具体做法如下: 首先从小到大枚举 L 。对于给定的 L , 先只考虑 $0 < p < L$, 计算出所有 $d(i, L)$ 的中间结果, 然后把这些 $d(i, L)$ 从小到大排序。对于每个 $d(i, L)$, 看看是否有状态 j 满足: $d(j, 0)$ 有定义 (此时它肯定是空串), 并且存在规则 $S \rightarrow ij$ 或者 $S \rightarrow ji$ 。如果存在, 用 $d(i, L)$ 更新 $d(S, L)$ 。这个过程类似第 11 章中将要介绍的 Dijkstra, 请读者仔细体会。

例题 9-31 送匹萨 (Pizza Delivery, ACM/ICPC Daejeon 2012, UVa1628)

你是一个匹萨店的老板, 有一天突然收到了 n 个客户的订单 ($n \leq 100$)。你所在的小镇只有一条笔直的大街, 其中位置 0 是你的匹萨店, 第 i 个客户的家在位置 p_i 。如果你选择给第 i 个客户送餐, 他将会支付你 $e_i - t_i$ 元, 其中 t_i 是你到达他家的时刻。当然, 如果你到的太晚, 使得 $e_i - t_i < 0$, 你可以路过他家但是不进去给他送餐, 免得他反过来找你要钱。

你只有一个送餐车, 因此只能往返地送餐, 如图 9-28 所示就是一个路线。图中的第一行是位置, 第二行是 e_i 。图上的路线对应的总收益为 12 (c_4 付 3 元, c_2 付 3 元, c_5 付 5 元, c_1 付 1 元)。

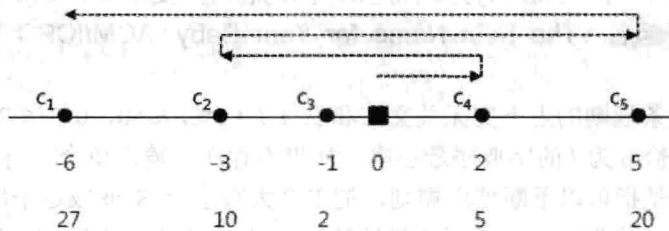


图 9-28 送餐路线

不过图 9-28 所示路线并不是最优的。最优路线是 $0 \rightarrow c_3 \rightarrow c_2 \rightarrow c_1 \rightarrow c_5$, 总收益是 32。你